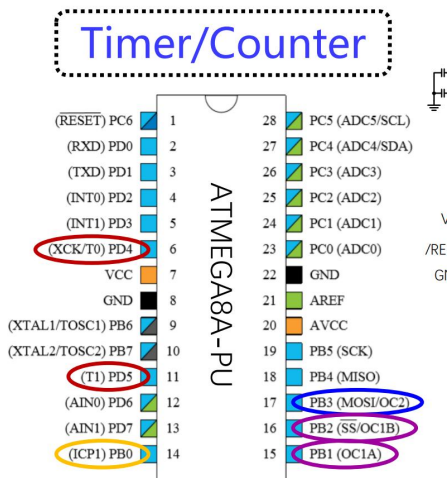


MCU 定时器/计数器与 PWM、直流电机、 步进电机及其编程实验思考总结

指导老师：李玉虎

一、简要介绍

本次首先介绍了 ATmega8A 的一些定时器/计数器，利用其第二功能实现，由图可见，其在 PD4, PD5/PB0, 等位置具有此功能可以加以实现。



二、定时器中断与 LED 控制（TC0 实验）

核心思考：定时器的本质是硬件实现的“时间沙漏”，通过中断机制解放 CPU 资源。TC0 的 8 位特性决定了其最大计时周期受限，需合理选择分频系数。

CS0[2:0]	TC0时钟	CS0[2:0]	TC0时钟
100	$CLK_{IO}/256$	000	无, 停止
101	$CLK_{IO}/1024$	001	CLK_{IO}
110	T0, 下降沿	010	$CLK_{IO}/8$
111	T0, 上升沿	011	$CLK_{IO}/64$

利用 CS0[2:0] 调整不同的值，可以实验其不同的时钟计数。

```
1 #include <avr/io.h>
2 #include <avr/interrupt.h>
3
4 // 添加中断服务程序声明
5 ISR(TIMERO_OVF_vect)
6 {
7     PORTC ^= (1 << PORTC0); // LED状态切换
8 }
9
10 int main(void)
11 {
12     DDRC |= (1 << DDC0); // 正确设置PC0为输出
13     PORTC &= ~(1 << PORTC0); // 初始化为低电平
14
15     TCNT0 = 0; // 计数器从0开始
16     TIMSK |= (1 << TOIE0); // 允许定时器溢出中断
17
18     // 配置定时器：1024分频，普通模式
19     TCCR0 = (1 << CS02) | (1 << CS00);
20
21     sei(); // 开启全局中断
22
23     while (1)
24     {
25         // 主循环保持空转
26     }
27 }
```

其中配置 PC0 关键：

```
// TC0 中断配置关键代码 ISR(TIMERO_OVF_vect) { PORTC ^= (1 << PORTC0);
// 耗时约 2 μs} void tc0_init() { TCCR0 = (1 << CS02) | (1 << CS00);
// 1024 分频 TIMSK |= (1 << TOIE0); sei(); }
```

三. 硬件 PWM 实现呼吸灯（TC1 模式 8）

关于 TC1 的时钟配置，其中有快速 PWM 模式和非 PWM 模式，二者的功能有些许相反。

PWM 模式：TC1 支持多种 PWM 工作模式，包括普通 PWM、相位校正 PWM 和快速 PWM。在本实验中，我们使用了快速 PWM 模式，该模式具有较高的频率和灵活性。

CS12	CS11	CS10	说明
0	0	0	无时钟源(定时器停止)
0	0	1	clk _{I/O} (没用预分频)
0	1	0	clk _{I/O} /8(用预分频)
0	1	1	clk _{I/O} /64(用预分频)
1	0	0	clk _{I/O} /256(用预分频)
1	0	1	clk _{I/O} /1024(用预分频)
1	1	0	外部时钟源T1,下降沿
1	1	1	外部时钟源T1,上升沿

`TCCR1A |= (1 << COM1A1);` //设置输出比较管脚切换方式等
`TCCR1B |= (1 << WGM13) | (1 << CS12);` //设置PWM模式为8, 时钟256分频

CS1[2:0]	TC1时钟
100	$\text{CLK}_{\text{IO}}/256$

定时器/计数器1的计数比较输出功能设置好以后, 自由工作, 不占用CPU时间

1. 设置使其 CS1 为 100。设置 OCR1A 寄存器的值, 以确定 PWM 信号的高电平持续时间。
2. 主函数中启动计数器并进入无限循环。

四. 用 TC1 的 1 路比较输出控制 1 个 LED 呼吸灯, 另 1 路的控制 1 路 LED 灯闪烁

1. 配置 TC1 为快速 PWM 模式, 并设置适当的时钟源和预分频值。
2. 初始化 I/O 端口, 将 PB1 和 PB2 分别设置为输出引脚。
3. 设置 OCR1A 和 OCR1B 寄存器的值, 以确定两路 PWM 信号的高电平持续时间。
4. 主函数中启动计数器并进入无限循环。

核心突破: 理解相位与频率校正 PWM 模式的双缓冲机制, 通过 ICR1 设定 PWM 周期, OCR1A 控制占空比。

TC1 计数说明:

1. TCNT1 计数方式:

加计数: TCNT1 从 0 增加到 ICR1。

减计数: TCNT1 从 ICR1 减少到 0。

2. A/B 路比较器:

当 $\text{TCNT1} == \text{OCR1A/B}$ 时 (加计数时), 输出 $\text{OC1A/B} = 1$ 。

当 $\text{TCNT1} == \text{OCR1A/B}$ 时 (减计数时), 输出 $\text{OC1A/B} = 0$ 。

3. TC1 计数溢出:

当 TCNT1 计数达到最大值时, 会产生溢出 (Bottom), 此时会触发 TOV1 溢出中断。

4. TOV1 溢出中断服务程序 (ISR):

在 TOV1 ISR 中, 改变 OCR1A/B 的值, 以调整下一次 OC1A/B 的匹配输出时间 (即高/低电平的持续时间)。

```
// 呼吸灯配置代码 void pwm_init() {    ICR1 = 2000;                // PWM  
周期= (2 * 2000) / (1MHz/8) = 32ms    OCR1A = 500;                // 初始占
```

```
空比 25%    TCCR1A = (1 << COM1A1); // 非反向模式    TCCR1B = (1 << WGM13)
| (1 << CS11); // 模式 8, 8 分频
```

五. 直流电机 PWM 调速系统

关键问题：发现低速时电机抖动现象，通过修改 PWM 模式解决：

```
// 改进的电机驱动配置 TCCR1B = (1 << WGM13) | (1 << CS11) | (1 << CS10);
// 模式 8, 64 分频 ICR1 = 625; // PWM 频率= 1MHz/(64*(2 * 625))=12.5Hz
```

- 实验步骤：
- 1 配置 TC1 为 PWM 模式，并设置适当的时钟源和预分频值。
 - 2 初始化 I/O 端口，将 PB1 设置为输出引脚。
 - 3 设置 OCR1A 寄存器的值，以确定 PWM 信号的高电平持续时间，从而控制电机速度。
 - 4 主函数中启动计数器并进入无限循环

六、步进电机精确控制

- 实验步骤：
- 1 配置 TC1 为输出比较模式，并设置适当的时钟源和预分频值。
 - 2 初始化 I/O 端口，将 PC3、PC2、PC1 和 PC0 分别设置为输出引脚。
 - 3 编写步进电机控制函数，根据输入参数生成相应的脉冲序列。
 - 4 主函数中调用步进电机控制函数，并根据按键输入调整电机方向。
- 思考与总结：
- 步进电机控制：通过精确控制脉冲序列，可以实现步进电机的精确定位和速度控制。
- 脉冲生成：TC1 的输出比较功能非常适合用于生成精确的脉冲信号，适用于步进电机控制。

MCU与28BYJ-48驱动板的连接	
电机驱动板	MCU
IN1	PC3
IN2	PC2
IN3	PC1
IN4	PC0
+	VCC(+)
-	GND(-)

五、深度优化思考

实验感悟：

我们深入了解了 ATmega8A 微控制器的定时器/计数器模块的功能和应用。从简单的 LED 控制到复杂的电机控制，我们掌握了如何使用定时器/计数器来实现各种自动化任务。

通过寄存器级编程深刻理解了硬件工作原理，但未来实际开发中应权衡寄存器操作与库函数的使用。电机控制本质上是对能量和时序的精确把控，需要将软件算法与硬件特性紧密结合。后续拟结合编码器实现真正闭环控制，探索更复杂的运动控制算法。深入了解了 ATmega8A 微控制器的定时器/计数器模块的功能和应用。从简单的 LED 控制到复杂的电机控制，我们掌握了如何使用定时器/计数器来实现各种自动化任务。